SOMAIYA VIDYAVIHAR UNIVERSITY

Internal Assessment Task II - **Agentic AI System using Modern LLM-based Tools**

Research Assistant Agent using a ReAct LLM Agent with Tool Orchestration

by

16015024046: Sheersh Nagaich
16015024047: Radhesh Nair
16015024048: Sohan Narewadi

For the subject

Artificial Intelligence

Department of Electronics & Computer Engineering
Academic Year 2026-27

Research Assistant Agent using a ReAct LLM Agent with Tool Orchestration

Group of 3, same lab batch | Demo to be taken in lab duration

1. Introduction and Abstract

Classical agents studied in the syllabus (simple reflex and model-based agents) choose actions from fixed condition-action rules. Modern agentic AI replaces the hand-written rules with a Large Language Model (LLM) that reads a goal, decides what to do next, calls external tools, observes the result and repeats until the goal is met. The focus is therefore on tool usage, reasoning and orchestration, not on model training.

This project builds a Research Assistant Agent. The user gives a goal in plain English, for example "read the campus AI report and compute the cost per student". The agent plans the steps, reads local documents and CSV files, uses a calculator for exact arithmetic, searches the web for general facts, and returns a cited answer. Every step (Thought, Action, Observation) is logged for inspection. We also implement a classical rule-based agent and run both on the same goals to compare them (Part C).

AI framework(s) used: ReAct-style agent written from scratch in Python (Thought, Action, Observation loop, no heavy framework, so every step is visible and explainable); OpenRouter API (OpenAI-compatible, free tier) for LLM access via the openai Python SDK.

Tools integrated with agent: (1) Calculator (safe AST evaluator), (2) Document reader (txt/pdf) with a document lister, (3) CSV query tool (simple database), (4) Web search (Wikipedia search API).

2. Part A: Agent Design (Modern View)

Element	Design decision
Goal of the agent	Answer multi-part research questions by combining facts from local documents, data tables and the web, with exact numeric results and source citations.
Agent role	Autonomous research assistant: it decomposes the goal, chooses tools, interprets observations, recovers from errors and decides when it has enough information to answer.
Tools used	calculator, list_documents, read_document, csv_query, web_search (details in Part B).
Memory type	Short-term / context memory: the full message history (system prompt, goals, thoughts, observations) is resent to the LLM at each step and kept across user turns in one session.
Planning strategy	ReAct (Reason + Act): at each step the LLM writes a Thought, picks one Action with an input, receives a real Observation, and repeats (max 10 steps) until it writes a Final Answer.
Environment (PEAS)	Performance: answer correctness, correct tool use, few steps. Environment: local docs folder, CSV files, Wikipedia. Actuators: tool calls + answer text. Sensors: tool observations and user text.

3. Part B: Implementation (Modern AI Tools)

3.1 LLM-based agent

The agent is one LLM that acts as the planner and reasoner. We call NVIDIA Nemotron-3-Super (120B parameters, 12B active) through OpenRouter's free tier, with automatic fallback to Gemma-4-31B and Nemotron-3-Ultra if a model is rate-limited or unavailable. Reasons for this choice: (a) it is free, so the project needs no paid credits; (b) the model is large enough to follow the strict Thought/Action/Observation text format; (c) OpenRouter exposes an OpenAI-compatible API, so the model can be swapped by changing one string without touching the agent code. Temperature is 0.1 to keep tool decisions stable. The system prompt (Fig. 7) defines the tools, the exact output format and the rules (never invent facts, use the calculator for all arithmetic, one tool per step).

3.2 Tools used

Tool	What it does	Why chosen / features
calculator	Evaluates arithmetic expressions	LLMs make arithmetic errors; the tool is exact. Uses Python ast with a whitelist of operators, never eval(), so it is safe against code injection.
list_documents / read_document	Lists and reads .txt and .pdf files in docs/	Gives the agent private knowledge it was not trained on (document reader). PDF via pypdf; output truncated at 4000 chars; path sanitised with basename.
csv_query	Previews a CSV or returns count/sum/mean/min/max of a numeric column	Acts as a simple database tool (CSV/JSON option in the brief).
web_search	Wikipedia search API, plus intro paragraph of the top hit	Brings in open-world facts; no API key needed, so the demo is reproducible by anyone.

3.3 Multi-step reasoning

The agent never answers from a single call. In Goal 3 (see logs) it first previewed the CSV to learn the column names, then called the calculator twice (mean adoption, score improvement), then searched the web, noticed the first search returned irrelevant pages, reformulated the query, and only then wrote the answer. This sequence was chosen by the LLM at run time; nothing in the code fixes it. Orchestration code in agent.py handles four things:

- **Parse and dispatch:** parses the LLM text (also tolerates JSON-style actions), validates the tool name, runs the tool, and feeds back the Observation.
- **Hallucination guardrail:** the LLM sometimes ignores the stop sequence and invents its own Observation. In our first run it fabricated a CSV with wrong columns and numbers and answered from it. agent.py now cuts the output at the first "Observation:", so only real tool output is used.
- **Error recovery:** invalid format or unknown tool returns an ERROR observation and the LLM corrects itself on the next step.
- **Step limit:** max 10 steps prevents infinite loops.

3.4 Architecture diagram

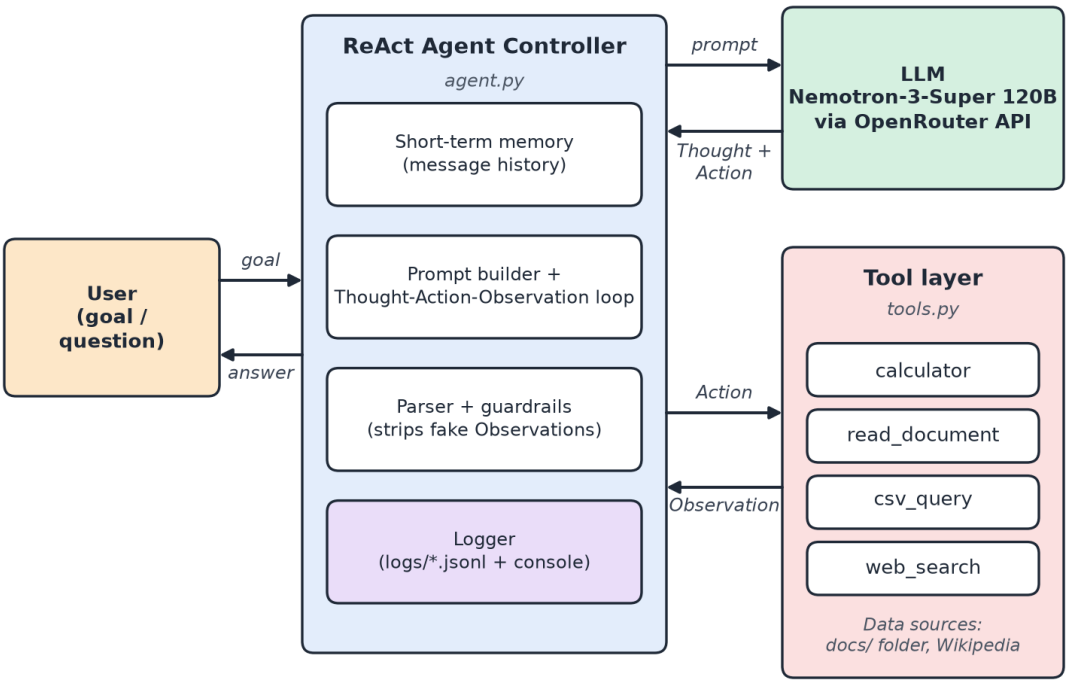


Fig. 1: Architecture. The controller keeps memory, calls the LLM, validates the Action and dispatches to tools; every step is logged.

3.5 Tool flow diagram

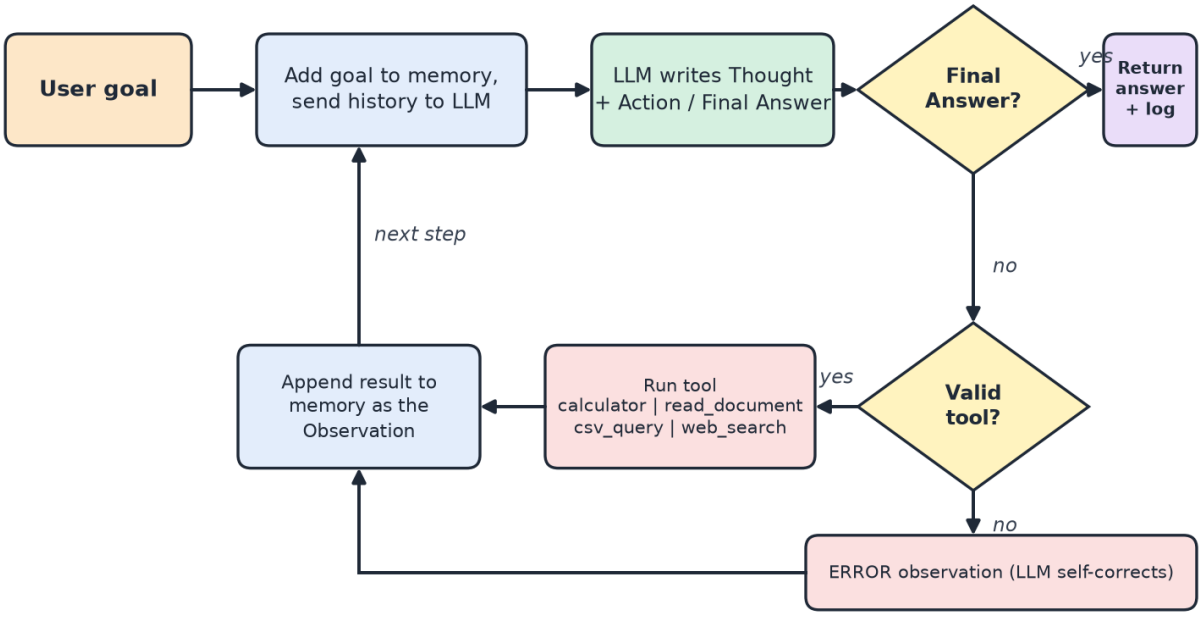


Fig. 2: Tool flow of one goal: loop until the LLM emits a Final Answer or the step limit is reached.

3.6 Observable agent behaviour (logs / traces)

Every run writes a JSON-lines trace (logs/demo_goalN.jsonl) with a timestamp and one record per event: goal, llm (model + full Thought/Action text), tool (tool name, input, observation) and final. The same

trace is printed live on the console, and logs/demo_transcript.txt holds the whole session. The table summarises the four demo runs; the screenshots follow.

Goal	LLM steps	Tools called (in order)	Result
1 Read report, find main finding + top department	3	list_documents, read_document	68% weekly use; ECE highest at 81% (correct)
2 Cost per student vs Rs. 250 target	2	calculator	Rs. 200.0, below target (correct)
3 CSV statistics + web definition	6	csv_query, calculator, calculator, web_search, web_search	Improvement 7.5, mean adoption 64.4% (both correct); definition weakly sourced
4 Follow-up using memory only	1	(none needed)	Recalled Rs. 200.0 from earlier turn

3.7 Code repository / notebook

Files in project/: agent.py (ReAct loop, LLM client, logging), tools.py (4 tools), classical_agent.py (rule-based baseline), run_demo.py (runs the demo goals on both agents), make_assets.py and build_report.py (figures and this report), docs/ (sample report and CSV), logs/ (interaction logs). Run: set OPENROUTER_API_KEY, then python run_demo.py, or python agent.py for an interactive chat that keeps memory across questions.

3.8 Agent interaction logs, prompts and output screenshots

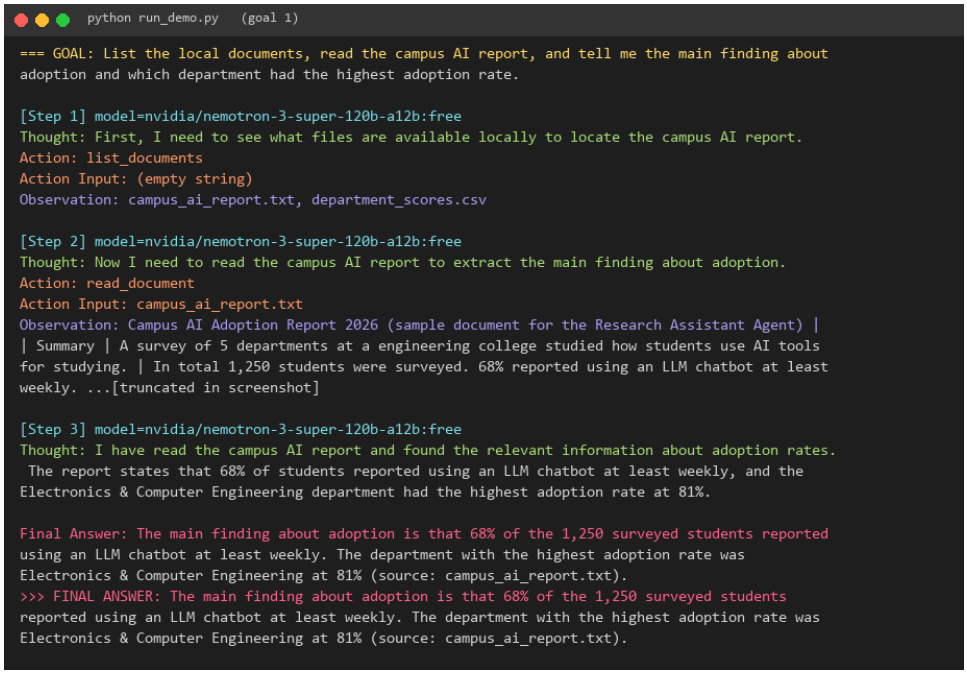


Fig. 3: Goal 1 trace. Two tool calls, then a cited Final Answer.

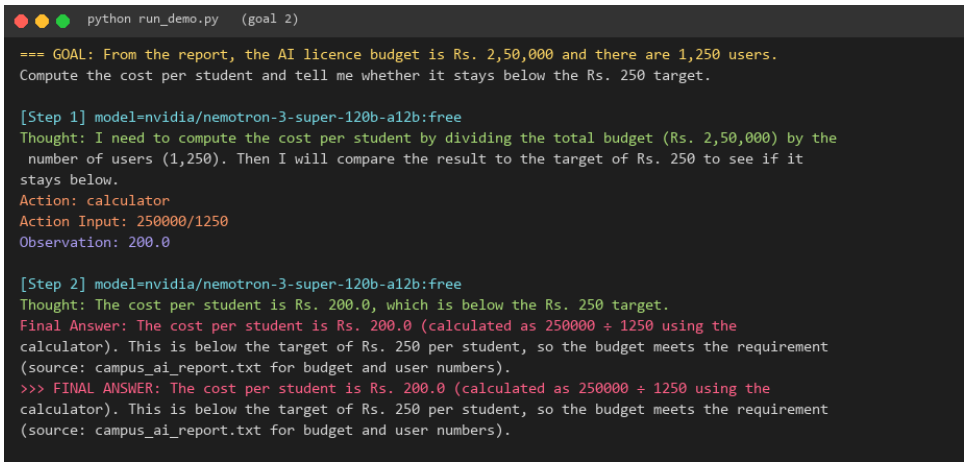


Fig. 4: Goal 2 trace. The agent uses the calculator tool instead of doing the division itself.

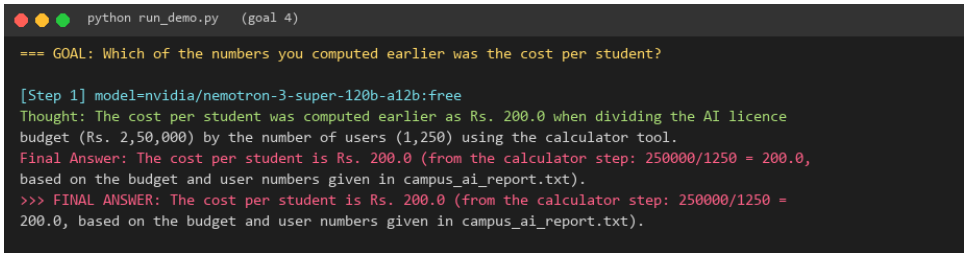


Fig. 5: Goal 4. A follow-up answered from short-term memory without any tool call.

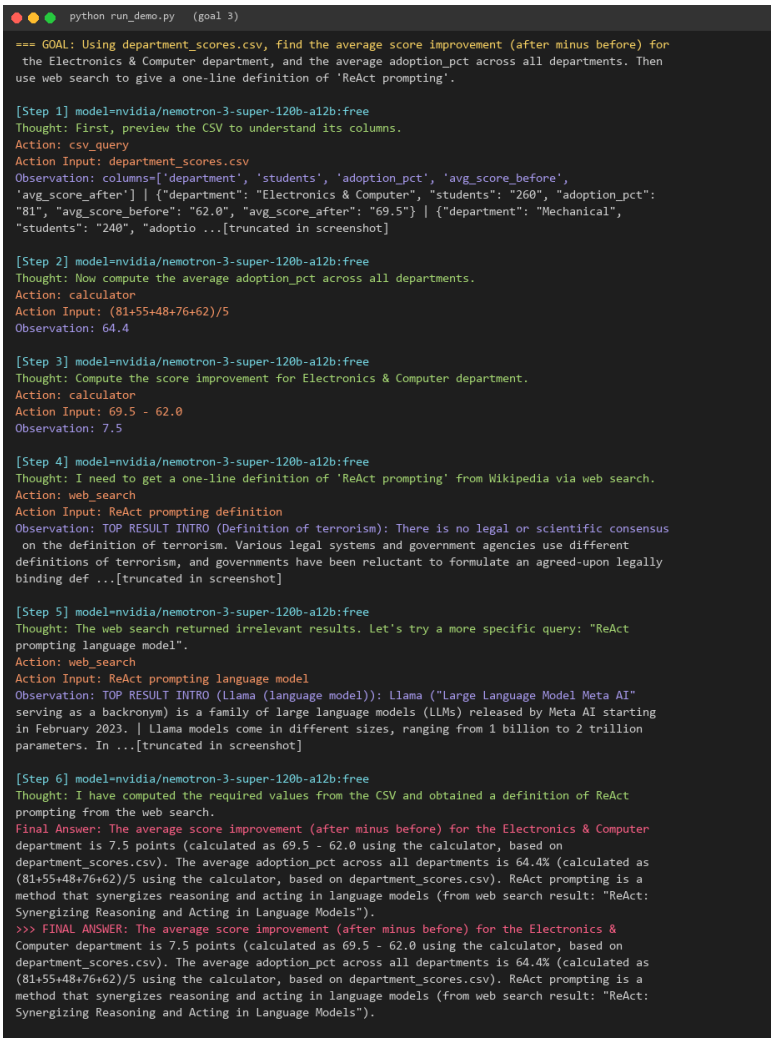


Fig. 6: Goal 3 trace. Five tool calls, including a re-planned web search after an irrelevant result.

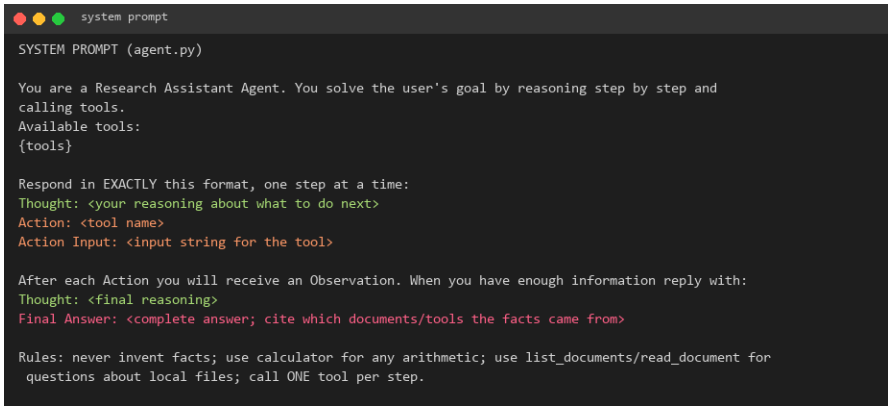


Fig. 7: System prompt given to the LLM.

4. Part C: Analysis and Comparison

To compare fairly, classical_agent.py implements a simple reflex agent: an ordered list of keyword/pattern rules, the first matching rule fires, no LLM, no memory. We gave it the same three goals (Fig. 8).

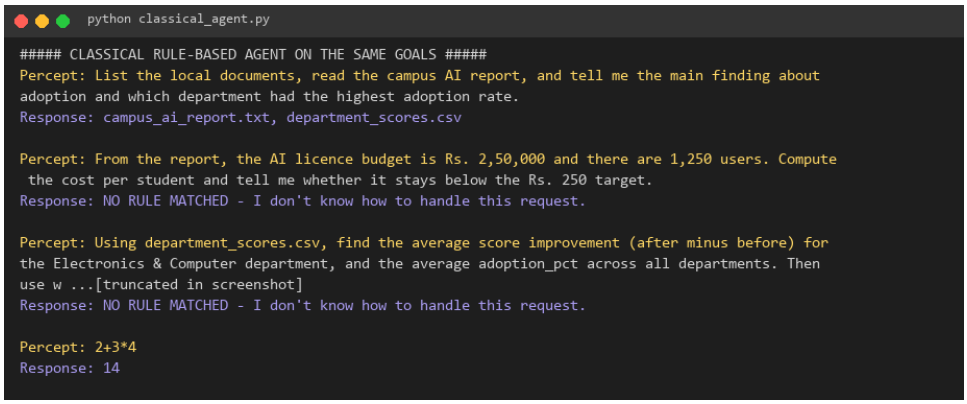


Fig. 8: Classical rule-based agent on the same goals. It can only run a rule that matches a fixed pattern.

Aspect	Classical agent (syllabus)	Modern agent (this activity)	Evidence from our runs
Decision logic	Rule-based, hard-coded	LLM-driven	Classical failed 2 of 3 goals (NO RULE MATCHED); LLM agent completed 3 of 3
Planning	Static logic	Dynamic planning	Goal 3: LLM re-planned after a bad search and used 5 tool calls
Memory	None	Contextual memory	Goal 4: recalled Rs. 200.0 without re-computing
Reasoning depth	Single step	Multi-step reasoning	Classical answers 1 percept with 1 action; LLM chained 1-5 tool calls
Input	Exact patterns only	Natural language	Goal 1 triggered only the 'list documents' rule in the classical agent and never read the report
Reliability	Deterministic, always same output	Probabilistic; can hallucinate	LLM invented an Observation in our first run (fixed by guardrail)
Cost / speed	Instant, free	Several API calls per goal	LLM goals took 1-6 LLM calls each
Explainability	Rule that fired is obvious	Thought text + logs	Thought lines make each decision auditable

5. Discussion and Takeaways

- **Flexibility:** The LLM handled phrasing it was never programmed for, which is the main gain over rule-based agents.
- **Orchestration matters as much as the model:** the model is right to call a calculator but can still misuse tools. We saw it wrap actions in JSON, drift from the format, and fabricate an Observation. Parsing tolerance and the guardrail turned these into recoverable events.
- **Limitation found:** in Goal 3 the Wikipedia search returned irrelevant pages (terrorism, Llama) and the final definition of ReAct was only loosely supported by the search output (it quoted the paper title). A stronger search API (e.g. Tavily/Serper) and a rule that the agent must say "not found" when evidence is missing would fix this. We report it as a limitation rather than hide it.
- **Operational lesson:** free models can be rate-limited or removed (all three of our original default model names had been retired by the time we ran it), so the fallback list is necessary.

- **When to use which:** a deterministic rule-based agent is better for fixed, safety-critical tasks; the LLM agent is better for open-ended tasks. A hybrid (LLM for planning, rules for validation) is the practical design.

6. Conclusion

We built a working LLM-based ReAct agent with four tools, short-term memory, multi-step planning, error recovery and full JSONL traces, and compared it with a classical rule-based agent. On three realistic goals the modern agent produced correct, cited answers (including exact arithmetic through the calculator tool) while the classical agent could solve only the one goal that matched a hard-coded rule. The project shows that agentic AI extends classical agent ideas (perceive, decide, act) with dynamic planning and contextual memory, and that reliable behaviour depends on careful tool design and guardrails around the model.

Future work: a stronger search tool, a Google Calendar tool (OAuth 2.0) for scheduling, long-term memory with a vector store, and an automatic evaluation set.

7. References

[1] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR 2023. arXiv:2210.03629.

[2] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020 (agent types, PEAS).

[3] OpenRouter API documentation, https://openrouter.ai/docs (OpenAI-compatible chat completions).

[4] MediaWiki Action API, https://www.mediawiki.org/wiki/API:Main_page (web_search tool).

[5] Python ast module documentation, https://docs.python.org/3/library/ast.html (safe calculator).